

What Google Knows: Measuring Telemetry and Identifier Exposure Across Privacy-Focused Android Operating Systems

Bruno Mirčevski

Faculty of Computer Science, Bialystok University of
Technology
Bialystok, Poland

Krzysztof Jurczuk

Faculty of Computer Science, Bialystok University of
Technology
Bialystok, Poland

Abstract

This paper presents a comparative analysis of user privacy in Android operating systems through network traffic inspection. Using a controlled man-in-the-middle setup and traffic decryption techniques, six Android systems were evaluated: stock Android (GMS), GrapheneOS, iodeOS, and three variants of LineageOS (stock, Gapps, microG). While prior work focuses on applications or stock Android across device manufacturers, this study investigates alternative Android distributions designed with privacy as a primary objective. We examined outbound connections, transmitted identifiers, and network traffic patterns to assess the impact of integrated Google services on user privacy.

Keywords

Android, Privacy, Network traffic

1 Introduction

Android is the dominant mobile operating system and acts as a primary gateway to the digital world for billions of users. It is crucial for the core operating system to be a reliable, stable and trustworthy platform that allows users to run applications and services of their choice. While third-party application privacy has been widely studied [1, 6, 7, 9], less attention is typically paid to the operating system layer and to privileged service frameworks that silently enable additional functionality.

While the Android core is the Android Open Source Project (AOSP), most consumer devices include GMS (Google Mobile Services) layered on top. They are implemented as privileged system applications to provide baseline functionality. Application developers commonly assume their presence by default. This design tightly couples many functions into a single service stack. Enabling useful capabilities may implicitly enable others that a user would not choose, such as advertising identifiers, telemetry, and behavioral tracking. Because Google's business model is strongly advertising-driven, many users may be unwilling to entrust this provider with broad access to their devices and data.

In principle, users should be able to choose which services they rely on and have confidence that these choices are enforced by the platform's permission and isolation mechanisms. This demand for transparency and controllability motivates alternative approaches such as microG¹ and GrapheneOS sandboxed Google Play². These solutions aim to preserve device functionality and application compatibility while reducing privilege and improving user control over Google-related components.

Privacy-focused Android operating systems aim to reduce data exposure at the OS level. They achieve this by limiting preinstalled privileged components, tightening default settings, and providing more transparent controls. We focus on three representative approaches: GrapheneOS³ prioritizes a hardened security model and strong isolation. Google components are not included by default, but can be installed in a sandboxed mode as ordinary applications. LineageOS⁴ provides a lightweight, broadly supported AOSP-based system with minimal preinstalled software. It can be deployed in multiple configurations: without Google services, with proprietary Google Play Services (Gapps⁵), or with microG⁶. Finally, iodeOS⁷ is a privacy-oriented distribution that combines a de-Google baseline with additional privacy features and preconfigured app ecosystem choices, shipping with microG.

Prior work has established that Android privacy risks arise not only from user-installed applications, but also from the operating system and its preinstalled components [2–5, 8]. Official Android builds transmit substantial data to OS vendors and third parties, including telemetry, location, app activity, and persistent identifiers. Google serves as the dominant driver of this background tracking ecosystem. Prior literature shows that removing or replacing these components with open-source variants drastically reduces unsolicited communication [4]. However, alternative distributions remain under-explored in reproducible measurement studies. This motivates our systematic comparison of stock and privacy-oriented systems.

Our main contributions are as follows: first, we present a comparative network-level privacy analysis of six Android configurations across diverse Google integration models; second, we provide the first systematic empirical comparison between privileged GMS, sandboxed Google Play Services, and microG under identical conditions; third, we design a controlled, real-device traffic interception and TLS decryption measurement methodology tailored for Android 16; and fourth, we quantify how alternative distributions reduce background data and identifier exposure while preserving core functionality.

2 Experiment design

2.1 Hardware and software

We used a single Google Pixel 6a device (codename *bluejay*) to evaluate the studied Android distributions. All tested builds are based on Android 16 and were obtained in December 2025: Stock Android

¹<https://github.com/microg/GmsCore/wiki>

²<https://grapheneos.org/usage#sandboxed-google-play>

³<https://grapheneos.org/>

⁴<https://lineageos.org/>

⁵<https://mindthegapps.com/>

⁶<https://lineage.microg.org/>

⁷<https://iode.tech/iodeos/>

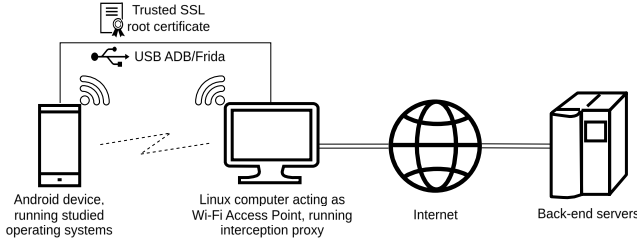


Figure 1: Measurement setup. The Android device connects to a Wi-Fi access point hosted on a Linux computer that runs traffic interception and capture software. A custom system certificate is installed on the phone using root access. A USB connection is used for ADB/Frida communication.

(BP4A.251205.006); LineageOS (lineage-23.0-20251206, BP2A.250805.005); LineageOS for microG (lineage-23.0-20251203-microG, BP2A.250805.005); GrapheneOS (2025121200, BP4A.251205.006); iodeOS (7.0-20251129-bluejay, BP2A.250805.005).

Network traffic analysis on Android is challenging due to enforced encryption, certificate pinning, and anti-tamper mechanisms. Traffic was captured using a controlled man-in-the-middle (MITM) gateway with a Linux host acting as a Wi-Fi router, redirecting HTTP(S) traffic through an intercepting proxy. QUIC over UDP/443 was blocked to force TCP-based TLS. The proxy CA was installed as a system-trusted certificate. We performed runtime certificate unpinning via Frida using scripts from HTTP Toolkit⁸. Full packet captures were recorded and compared across distributions. Where needed, we used publicly available Protocol Buffer definitions from microG’s source code⁹ to decode payloads. We acknowledge that we were not able to capture and analyze all traffic. Instead, we focus on identifying systematic differences between Android distributions and general behavioral patterns. A preview of the measurement setup is shown in Fig. 1.

2.2 Measurement scenarios

To enable consistent and comparable measurements across Android distributions, four scenarios were defined. Scenarios were adapted to each system’s capabilities (or skipped if not applicable). As a general principle, any required configuration step not available during initial setup was performed as soon as possible thereafter.

Scenario A1: Default setup with Google account. Typical user path. Google account sign-in is performed during setup and default recommendations are accepted (including prompted permissions).

Scenario A2: Minimal setup with Google account. This scenario extends *Scenario A1* by adjusting settings towards privacy while retaining Google account functionality. During and after initial setup, all optional consents and permissions are declined, and settings are adjusted to maximize privacy. For privacy-focused systems, the distinction between Scenario A1 and Scenario A2 is minimal; therefore, we skipped it.

Scenario B: Minimal setup without Google account. Maximum privacy in mind, avoiding Google account sign-in entirely. Only

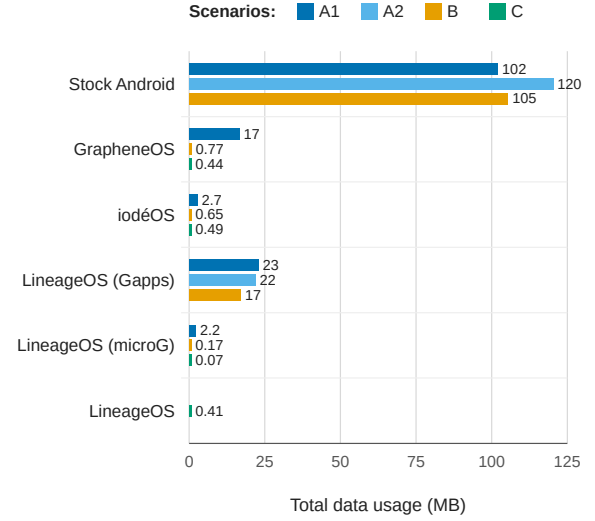


Figure 2: Total data usage aggregated across all measurement tasks. Scenario applicability varies by system.

strictly required prompts are accepted. Google components remain present and enabled, but operate without an account. This scenario applies to the same systems as Scenario A1.

Scenario C: Extremely minimal setup without Google components. Applies only to distributions that do not enable Google services by default. If microG is preinstalled, it remains disabled.

2.3 Measurement tasks

We performed the following tasks sequentially for each scenario. Network traffic was captured using Wireshark¹⁰ for basic analysis.

Task 0: Initial setup. The device was configured according to the target scenario. Network traffic capture began with the device’s first Internet connection and continued until 10 minutes after setup completion.

Task 1: System idle. The device was rebooted, then kept connected to the monitored network for at least 30 minutes. During this period, the device was unlocked three times and the application list was scrolled through. No applications were opened.

Task 2: Basic application interactions. Simple actions were performed: disable haptic feedback; toggle dark mode; check per-application battery usage; set an alarm; create a folder; create an on-device contact.

3 Results

Total data usage across all operating systems and scenarios, aggregated for all tasks is presented in Figure 2. In all scenarios stock Android shows the highest data usage. Generally, configurations incorporating more Google services generate more traffic.

Figure 3 presents the domain-level distribution of network traffic. Stock Android contacts the most Google-related domains. iodeOS and LineageOS contact fewer servers overall. iodeOS also relies on some Mozilla infrastructure not present in other systems.

⁸<https://github.com/httptoolkit/frida-interception-and-unpinning>

⁹<https://github.com/microg/GmsCore/>

¹⁰<https://www.wireshark.org/>

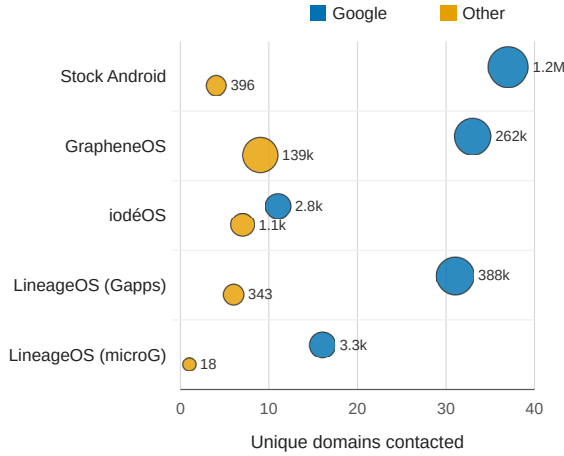


Figure 3: Domains contacted in Scenario A1 aggregated for all tasks, grouped by operating system and by Google-related and other endpoints. Bubble size represents the total number of packets exchanged with each group.

Table 1 shows the infrastructure dependencies for core system functions. LineageOS remain closest to stock Android, relying entirely on Google servers. iodeOS adopts an intermediate position, while GrapheneOS demonstrates the strongest de-Googling, substituting all three functions with its own or independently operated infrastructure.

Beyond aggregate traffic measurements, we were able to decrypt and analyze some payloads. A subset of these observations originates from earlier experiments performed on Android 14, as the Frida-based method became unsuccessful on Android 16.

Android’s check-in mechanism transmits real IMEI, MAC and hardware serial number to Google, while microG generates randomized, rotating identifiers (IMEI consistently prefixed with 35503104, MAC addresses with OUI b4:07:f9). The Android ID field in microG requests is frequently empty or zeroed. The Google Advertising ID (ADID) was observed in multiple requests on stock Android, formatted as a UUID. This value remained constant across different requests, confirming its persistence as a cross-application tracking mechanism. After a manual reset in system settings, the ADID was transmitted as all zeros, indicating the identifier was actually cleared. We observed requests to *app-measurement.com* on systems with Google services present, containing application package names, installation sources, versions, and the system theme preference.

When enabled, microG performs network geolocation without Google infrastructure by querying a selected provider, in our case *api.position.xyz*. The request transmits nearby cell towers and returns approximate coordinates. We compared Google Play Store with Aurora Store (a popular application source in alternative Android distributions) by downloading the same application from each platform. Both stores retrieved identical packages from the same Google CDN infrastructure. However, the Play Store generated 106 requests, while Aurora Store produced 93.

Table 1: Domains observed for connectivity check, remote provisioning, and location assistance.

System	Connectivity check	Provisioning	Location
Stock	connectivitycheck.gstatic.com	–	supl.google.com, agnss.google.com
Lineage	connectivitycheck.gstatic.com	remote.provisioning.googleapis.com	supl.google.com, agnss.google.com
iodéOS	captiveportal.kuketz.de	remote.provisioning.googleapis.com	supl.google.com
Graphene	connectivitycheck.grapheneos.network	remote.provisioning.grapheneos.org	gs-loc.apple, grapheneos.org

Table 2: Comparison of Google Takeout data across Android configurations.

System	Device IDs	Apps	Pixel telem.
Stock Android	IMEI, serial	Extensive	Full
Lineage (Gapps)	IMEI, serial	Extensive	Reduced
Lineage (microG)	–	–	–
GrapheneOS	Falsified serial	4 apps	–

In addition to network-level observations, Google Takeout¹¹ was used to download data associated with test accounts for different Google services implementations. For every logged-in system, a corresponding entry appeared in *devices.json*, including device model, OS version, country, language, registration timestamp, and last activity time. However, the depth of telemetry varied considerably across configurations, as summarized in Table 2.

4 Discussion

Our measurements reveal a clear hierarchy in how different Android configurations handle user data and privacy. The most fundamental finding is that adjusting privacy settings on stock Android, while beneficial, cannot provide the same level of protection as switching to an alternative distribution. On systems running proprietary Google services, privacy-adjusted scenarios showed only modest reductions in data transmission compared to the default setup. This is due to the underlying architecture, which tightly integrates Google components into the core of the OS and maintains persistent background activity. As a result, substantial data flows to Google infrastructure continue regardless of user-facing privacy settings.

Alternative distributions shift the privacy posture by making Google communication user-initiated rather than ambient. All privacy-focused systems generated orders of magnitude less network traffic with Google. Some of them utilize their own or third-party privacy-focused infrastructure to replace Google services. LineageOS with GMS was an exception: although it is more minimal, having official Google services preinstalled means it does not differ much from stock Android in terms of privacy. Among other configurations enabling basic Google functionality, microG represents the most privacy-preserving approach, though this comes with potential security trade-offs (signature spoofing, privileged install). Our measurements show that microG transmits the least data

¹¹<https://takeout.google.com/>

to Google infrastructure while maintaining basic service compatibility. It achieves this through identifier randomization rather than exposing authentic hardware identifiers, similar to GrapheneOS. Our observations illustrate the fundamental difference: microG optimizes for privacy, whereas GrapheneOS prioritizes security while still providing strong privacy, substantially improved over stock systems.

Alternative distributions also improve user control through minimal base systems and enhanced permission granularity. All tested alternatives ship without forced user agreements during initial setup, unlike stock Android, which requires accepting mandatory terms including data collection. A notable privacy-enhancing feature present in all tested systems, which is absent in stock Android, is per-application network permission control, allowing users to restrict Internet access for specific applications. Aurora Store demonstrates that downloading applications from official repositories remains viable without full Google infrastructure interaction, retrieving identical packages from the same source while generating fewer requests. Updates in alternative systems are typically smaller, more transparent, and user-controlled rather than automatic and opaque.

Several limitations qualify these findings. Our measurements capture specific, time-bounded scenarios rather than universal behavior. Traffic patterns likely vary with extended use and evolving service configurations. The TLS interception approach, while revealing substantial information, remains incomplete due to certificate pinning, anti-tamper mechanisms, and ongoing protocol evolution. GrapheneOS's resistance to our instrumentation, while advantageous for security, means we cannot closely verify what data sandboxed services transmit. These constraints indicate that our findings reflect general trends and architectural differences rather than definitive or absolute measurements.

Based on the network traffic we analyzed, Google may potentially distinguish between stock Android and alternative distributions through device characteristics transmitted during service interactions, raising the possibility of selective enforcement or access restrictions. Ultimately, the choice between privacy-focused alternative operating systems depends on individual threat models and functional requirements, but all of them represent meaningful improvements in user control and privacy over stock Android.

5 Conclusion

In this work, we compared network behavior across multiple Android distributions, measuring data transmission during setup, idle periods, and basic application usage. We evaluated stock Android, LineageOS variants, iodéOS, and GrapheneOS to quantify how different approaches to Google services affect user privacy. Our findings show that alternative distributions reduce unsolicited data flows by orders of magnitude compared to stock systems. These systems enhance privacy by reducing data exposure, limiting or randomizing identifiers, and using independent infrastructure. Network traffic and Google Takeout analyses revealed that stock Android transmits persistent hardware identifiers and various telemetry data, whereas alternative systems either do not transmit such data or significantly limit its scope.

Future work could extend measurements to longer usage periods and repeated trials to distinguish systematic differences from transient variation. More realistic tasks incorporating diverse applications would better approximate everyday use. As Android, Google services, and their alternatives continue to evolve, repeating this study would help determine whether future privacy improvements meaningfully alter the patterns we observed.

References

- [1] Haojian Jin, Minyi Liu, Kevan Dodhia, Yuanchun Li, Gaurav Srivastava, Matthew Fredrikson, Yuvraj Agarwal, and Jason I. Hong. 2018. Why Are They Collecting My Data? Inferring the Purposes of Network Traffic in Mobile Apps. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 2, 4, Article 173 (2018), 27 pages. doi:10.1145/3287051
- [2] Douglas J. Leith. 2023. What Data Do the Google Dialer and Messages Apps on Android Send to Google?. In *Security and Privacy in Communication Networks*, Fengjun Li, Kaitai Liang, Zhiqiang Lin, and Sokratis K. Katsikas (Eds.). Springer Nature Switzerland, Cham, 549–568.
- [3] D. J. Leith. 2025. *Cookies, Identifiers and Other Data That Google Silently Stores on Android Handsets*. Technical Report. Trinity College Dublin. https://www.scss.tcd.ie/doug.leith/pubs/cookies_identifiers_and_other_data.pdf
- [4] Haoyu Liu, Paul Patras, and Douglas J. Leith. 2021. Android Mobile OS Snooping by Samsung, Xiaomi, Huawei and Realme Handsets. (2021). https://www.scss.tcd.ie/doug.leith/Android_privacy_report.pdf
- [5] Haoyu Liu, Paul Patras, and Douglas J. Leith. 2023. On the data privacy practices of Android OEMs. *PLOS ONE* 18, 1 (2023), 1–15. doi:10.1371/journal.pone.0279942
- [6] Abbas Razaghpanah, Rishab Nithyanand, Narseo Vallina-Rodriguez, Srikanth Sundaresan, Mark Allman, Christian Kreibich, and Phillipa Gill. 2018. Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem. doi:10.14722/ndss.2018.23009
- [7] Jingjing Ren, Martina Lindorfer, Daniel J. Dubois, Ashwin Rao, David R. Choffnes, and Narseo Vallina-Rodriguez. 2018. Bug Fixes, Improvements, ... and Privacy Leaks - A Longitudinal Study of PII Leaks Across Android App Versions. In *Network and Distributed System Security Symposium*. <https://api.semanticscholar.org/CorpusID:4231807>
- [8] Douglas C. Schmidt. 2018. *Google Data Collection*. Technical Report. Digital Content Next (DCN). <https://digitalcontentnext.org/wp-content/uploads/2018/08/DCN-Google-Data-Collection-Paper.pdf>
- [9] Zhaohua Wang, Zhenyu Li, Minhui Xue, and Gareth Tyson. 2020. Exploring the Eastern Frontier: A First Look at Mobile App Tracking in China. In *Passive and Active Measurement (PAM 2020) (Lecture Notes in Computer Science)*, Anna Sperotto, Alberto Dainotti, and Burkhard Stiller (Eds.). Springer International Publishing, Cham, 314–328.